

Material Stream Modelling in OpenModelica



BY

VIRAJ DESAI, PROCESS ENGINEER

EMAIL: VIRAJ DESAI

LEVEL: IMMEDIATE

VERSION: 0.00

12-SEP-26

Preface

This guide picks up exactly where *Introduction to OpenModelica for Process Engineers* left off. That first guide deliberately kept chemistry out of the picture its purpose was to build comfort with OpenModelica itself: installing it, navigating OMEdit, understanding the Modelica language's basic grammar, and exporting a simulation's results. None of that involved a compound, a phase, or a flash calculation. This guide is where those two things finally meet.

Here, we move from a general-purpose modelling environment into an actual chemical process simulation library **OMChemSim** and use it to build something with real engineering content: a material stream capable of computing its own bubble point, dew point, and vapor-liquid split. Along the way, we'll do something the first guide didn't need to: work through genuine, real bugs in third-party open-source code, understand why they occur, and fix them properly rather than working around them. That troubleshooting material turned out to be some of the most valuable content in this guide, and it's presented that way deliberately, not apologetically.

By the end, you will be able to import OMChemSim into OpenModelica, build a multicomponent material stream from scratch, diagnose and reason about the errors you're likely to encounter along the way, combine OMChemSim with the Modelica Standard Library's own property models where appropriate, and export your results for further analysis the core competencies needed to keep going into unit operations and full flowsheets on your own.

About This Guide's Companion Library

OMChemSim is a free, open-source chemical process simulation library, developed by FOSSEE (Free and Open-Source Software for Education), a project of IIT Bombay. It extends OpenModelica's general-purpose modelling engine with domain-specific building blocks a process engineer needs material streams, thermodynamic packages, and unit operations built directly in the Modelica language and released under the permissive 3-Clause BSD License.

It is, by design, a smaller and younger effort than the commercial tools most process engineers already know. That's addressed directly and honestly in Chapter 1, and again throughout Chapter 5, rather than glossed over here the transparency this guide extends to its own subject matter is the same transparency open-source software offers its users in the first place: the freedom to inspect, question, and fix what doesn't yet work as intended.

Prerequisites

This guide assumes you've completed *Introduction to OpenModelica for Process Engineers*, or that you're already comfortable with what it covers: installing OpenModelica, navigating OMEdit, and writing a basic Modelica model by hand. Beyond that:

- Hardware: the same standard, modern Windows or Linux machine used for the first guide.
- Attitude: a willingness to read an error message carefully rather than treat it as a dead end Chapter 5 exists to make that easier, not to make errors disappear.

- Mindset: no prior chemical engineering software experience beyond what you already bring from your own background is required. If you've used DWSim or Aspen before, several sections point out exactly where that experience does and doesn't transfer.

Acknowledgements

The same thanks from the first guide apply here, and then some. To the global OpenModelica community and the Open-Source Modelica Consortium: the transparent, freely accessible environment you've built is what makes a guide like this possible at all. And specifically for this guide, a further thank you to FOSSEE and IIT Bombay, whose work building OMChemSim gave process engineers a genuine open-source path into chemical process simulation where none existed before. The issues documented in Chapter 5 don't diminish that effort they're offered in the same spirit of open collaboration that produced the library in the first place, in the hope that identifying them helps the project improve.

About the Author

Viraj Desai is an engineer who enjoys exploring open-source tools, learning new skills, and collaborating with the engineering community. This guide continues directly from his first, written while learning OMChemSim hands-on including the process of finding, understanding, and fixing the issues Chapter 5 describes.

Disclaimer

Important Notice: Educational Use Only

All exercises, models, and mathematical representations presented in this manual are strictly restricted to learning and educational purposes only.

The author is currently in an active learning phase and is continuously exploring the capabilities of OpenModelica and OMChemSim. The models contained within these pages are simplified representations of complex physical phenomena and are not intended for direct application in real-world industrial environments.

This guide also makes use of a modified copy of OMChemSim, patched by the author to address several issues encountered while building the models described here, and distributed under the same 3-Clause BSD License as the original (see [vendor/OMChemSim-v1.0/LICENSE](#) and [ATTRIBUTION.md](#) for full detail). These patches reflect the author's own diagnosis and are offered as-is; they are not officially reviewed, endorsed, or maintained by FOSSEE, IIT Bombay, or the Open-Source Modelica Consortium.

No responsibility is assumed by the author, FOSSEE, IIT Bombay, or the Open-Source Modelica Consortium for any loss, damage, or operational failure resulting from the use of the information, models, or patched code contained herein.

A Note to the Community: As this is a work-in-progress born from a learning journey, constructive feedback and cooperative insights are deeply appreciated on the guide itself, and on the patches proposed alongside it. Let us grow this knowledge base together.

For real-world design, operational decisions, and safety-critical applications, always consult qualified professional engineers and utilize validated, commercially supported software.

TABLE OF CONTENTS

1	What is OMChemSim?	8
1.1	What OMChemSim Is	8
1.2	What It Can Model	8
1.3	License and Attribution	9
1.4	Why This Guide Uses a Patched Version	9
2	Importing the Simulator Package in OpenModelica	10
2.1	What Readers Actually Need	10
2.2	Locating the Simulator Package	10
2.3	Loading It in OMEdit	11
2.4	Setting Up Your Own Workspace Model	12
2.5	Verifying the Import	12
3	Understanding the Key Building Blocks	13
3.1	Simulator.Streams.MaterialStream	13
3.2	Simulator.Files.ChemsepDatabase	14
3.3	Simulator.Files.ThermodynamicPackages	14
3.4	Simulator.Files.Interfaces.matConn	15
3.5	Simulator.GuessModels	15
3.6	Simulator.UnitOperations	15
4	Building a Process Model from Scratch Using the Simulator Package	16
4.1	Choosing a System	16
4.2	Writing MyFirstStream.mo	16
4.3	Line-by-Line Explanation	17
4.4	Check Model	18
5	Troubleshooting: Reading OpenModelica Errors	19
5.1	Why Errors Are Normal Here	20
5.2	How to Read a Translation Error	20
5.3	Common Errors You'll Hit	21
5.3.1	Broken imports	21
5.3.2	model vs. record specialization mismatch	21
5.3.3	Missing partial on template/mixin classes	21
5.3.4	Cross-class variable scoping	21

5.3.5	Missing each on array-valued unit attributes.....	21
5.3.6	Nonlinear initialization failures.....	22
5.4	When to Suspect the Library vs. Your Own Code.....	22
6	Declaring Other Streams in the Same Model.....	22
6.1	Overview: Two Streams, Two Property Methods, One Model.....	23
6.2	Composites: Joining a Stream to a Property Package.....	23
6.3	A Hydrocarbon Stream via Peng-Robinson (HC).....	24
6.4	A Water Stream via the IAPWS-IF97 Steam Tables (W).....	25
6.5	Watch This: Unit Mismatches Between Libraries.....	25
6.6	Reference-State Caution.....	26
6.7	TwoStreams: Running Both Side by Side.....	26
6.8	Bonus: Walking the Operating Point Over Simulated Time (StreamSweep).....	27
7	Results Generation and Exporting Plots.....	28
7.1	Setting Up the Test Case.....	28
7.2	Understanding Simulation Output Files.....	29
7.3	Export Method 1: The Graphical Export (Plots).....	29
7.4	Export Method 2: The CSV (Comma Separated Values) Export.....	30
7.5	What to Actually Look At for a Material Stream.....	30
8	References and Source Codes.....	30

List of Figures

Figure 1	OMEdit Before Loading Up the Simulator Package.....	11
Figure 2	OMEdit After Loading Up the Simulator Package.....	11
Figure 3	OMEdit With Simulator Tree View.....	12
Figure 4	Simulator Package Extended.....	13
Figure 5	Code for Material Stream.....	16
Figure 6	MyFirstStream Model Code.....	18
Figure 7	Model Check.....	19
Figure 8	Errors in Openmodellica.....	20
Figure 9	Peng Robinson Material Stream.....	23
Figure 10	Raoult's Law Material Stream.....	24
Figure 11	Fix of Tg, Pg, and Xg.....	25
Figure 12	handling of 2 streams.....	26
Figure 13	2 streams operating point solved independently.....	27
Figure 14	Setting up time function.....	27

Figure 15 HC.xvap vs. HC.T parametric plot	29
--	----

List of Tables

Table 1 Thermo-Packages	14
Table 2 Code Line Explanation	17

1 WHAT IS OMChemSim?

In our first guide, we built familiarity with OpenModelica and the Modelica language on their own terms: generic blocks, a ramp function, the mechanics of checking and simulating a model. None of that involved a single molecule. That was deliberate, the language and the chemistry are two separate things to learn and conflating them early makes both harder. This chapter closes that gap. Before we write a single line of process-modelling code, you should know exactly what OMChemSim is, what it can and cannot do for you, and why the copy of it we'll be using in this guide has been modified from what you'd download directly from its source.

1.1 WHAT OMChemSim IS

OpenModelica, as you saw in the first guide, is a general-purpose modelling and simulation environment. On its own, it knows nothing about vapor pressure, phase equilibrium, or the properties of ethanol. What it knows is how to solve systems of equations, any equations, from any domain. OMChemSim is what turns that general capability into a chemical process simulator: a library of pre-built Modelica classes, material streams, thermodynamic packages, and unit operations, that already contain the equations of chemical engineering, so you don't have to derive a bubble-point calculation or an energy balance from scratch every time you want to model a stream.

Think of the relationship the same way you'd think about DWSim or Aspen Plus: the underlying solver and the process-engineering objects you actually drag onto a flowsheet are two different layers, built by different efforts, at different times. OpenModelica is the solver layer. OMChemSim is the process-engineering layer sitting on top of it. It is developed and maintained by FOSSEE (Free and Open Source Software for Education), a project of IIT Bombay, as part of a broader effort to make free engineering tools available to students and practitioners who might otherwise only have access to expensive commercial licenses.

It's worth being upfront about scale, too. OMChemSim is a comparatively young, small library built primarily through student contributions rather than by a large dedicated team. That matters how you should approach it, and we'll come back to exactly what that means in Section 1.4 and again in Chapter 5.

1.2 WHAT IT CAN MODEL

At a high level, OMChemSim gives you three kinds of building blocks, each of which we'll look at properly in Chapter 3:

- **Material streams:** not simple mass-balance placeholders, but streams capable of a genuine VLE flash: given a temperature and pressure (or other valid specification pair), a stream will compute its own bubble point, dew point, and vapor/liquid phase split.
- **Thermodynamic packages:** the equations that tell a stream *how* to relate composition, temperature, and pressure to phase behavior. OMChemSim ships several: Raoult's Law, NRTL, UNIQUAC, UNIFAC, Peng-Robinson, and Grayson-Streed, each suited to different

kinds of systems (ideal mixtures, non-ideal liquids, hydrocarbons and light gases, and so on).

- **Unit operations:** heaters, coolers, mixers, splitters, valves, pumps, compressors, expanders, flash drums, reactors (CSTR and PFR), and distillation and absorption columns.

Underpinning all of this is a **compound property database** of 434 substances, derived from Chemsep data, everything from vapor pressure correlations to heat capacity polynomials, ready to be referenced by name rather than typed in by hand.

This guide focuses entirely on the first two of these three, streams and thermodynamic packages. Unit operations are a natural next step once you're comfortable here, but they build directly on everything in this guide, so we're deliberately keeping scope narrow.

1.3 LICENSE AND ATTRIBUTION

OMChemSim is distributed under the **3-Clause BSD License**, one of the most permissive open-source licenses available. In practical terms, this means you are free to use, modify, and redistribute it, including the modified copy this guide is built around, provided you keep the original copyright notice and license text intact, and don't use FOSSEE's or IIT Bombay's name to imply they endorse your particular fork. You'll find the original license text at [vendor/OMChemSim-v1.0/LICENSE](#) in this repository.

This isn't just a legal formality to skim past. If you go on to build your own OpenModelica projects using this library or a modified copy of it, as we have you are expected to carry this same attribution forward.

1.4 WHY THIS GUIDE USES A PATCHED VERSION

Here is the part of this guide we want to be completely transparent about. When we first loaded OMChemSim's upstream v1.0 release into a current version of OpenModelica (1.27.0) and tried to build the simplest possible model, a single material stream with two components, it did not work. Not because of anything we did wrong, but because the library itself contains several structural issues: a broken import statement, a mismatch between how certain classes are declared and how they're actually used, a missing keyword that affects nearly every unit operation in the library, a couple of missing attribute keywords on array-valued properties, and a set of default numerical starting guesses that only happen to work for whatever compounds the original authors tested with.

None of these are exotic edge cases. They sit in the core files, the material stream model, the shared initial-guess system, the compound property base class that every single example in the library depends on. This almost certainly reflects the library having been built and tested against an older, more permissive version of the OpenModelica compiler; today's compiler is stricter about several things the Modelica language specification always technically required.

Rather than treat this as a dead end, we've treated it as part of the learning material. Every one of these issues has been diagnosed, fixed, and documented in full technical detail in [vendor/OMChemSim-v1.0/ATTRIBUTION.md](#), which sits alongside the patched library in this

repository. Both the **RaoultsLaw.mo** and **PengRobinson.mo** thermodynamic packages used later in this guide have already had these fixes applied. We won't re-explain every one of these fixes here that's what Chapter 5 is for, and what the `ATTRIBUTION.md` file is for if you want the full detail on any single one. For now, the important thing to know is simply this: if you follow this guide using the patched library in this repository, you should not encounter these issues yourself. If you try to follow along using an unmodified copy from OMChemSim's own repository, you very likely will and Chapter 5 will tell you exactly how to recognize and fix each one if that happens.

2 IMPORTING THE SIMULATOR PACKAGE IN OPENMODELICA

With a clear idea of what OMChemSim is and why this guide's copy of it looks slightly different from what you'd download yourself, it's time to get it into OMEdit. This chapter is short and mechanical by design there's no chemistry here yet, just the plumbing needed before any of it can happen.

2.1 WHAT READERS ACTUALLY NEED

If you clone or download the full repository this guide lives in, you'll notice it contains a fair amount beyond the library itself: numbered example folders for each model built along the way, a **docs/** folder holding this guide and its predecessor, and a **tools/** folder with helper Python scripts for batch simulation runs. None of that is required to follow this guide.

The only thing you need is the Simulator package, found at **vendor/OMChemSim-v1.0/Simulator/**. Everything from here through the end of this guide depends on nothing else. If you're curious about the rest of the repository once you've finished, the project's root `README.md` explains what each of those other folders is for but for now, treat them as out of scope and move on.

2.2 LOCATING THE SIMULATOR PACKAGE

Inside the repository, the patched library lives under **vendor/OMChemSim-v1.0/**. The **vendor** folder name is a common convention in software projects: it signals "third-party code, kept deliberately separate from anything we've written ourselves," so that it's always clear which files are the library's and which are yours. Inside that folder, alongside the library's own **LICENSE** and **ATTRIBUTION.md** files, you'll find the **Simulator** folder itself and inside that, a file called **package.mo**.

That single file, **Simulator/package.mo**, is the entry point to the entire library. You won't open it and read through pages of code it's mostly a documentation and licensing wrapper but it's the file you'll point OMEdit at in the next step, and OMEdit will take care of pulling in everything else the library contains.

2.3 LOADING IT IN OMEDIT

With OMEdit open, go to **File → Open Model/Library File(s)** and navigate to **vendor/OMChemSim-v1.0/Simulator/package.mo**. Select it and confirm.

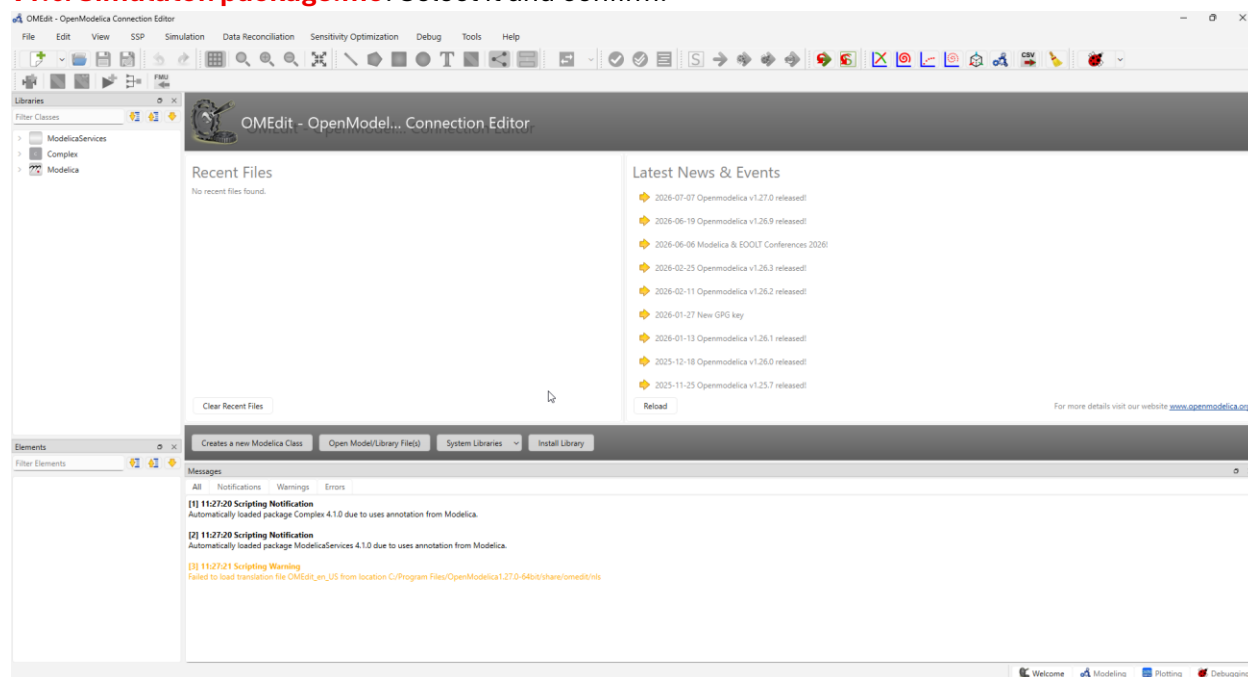


Figure 1 OMEdit Before Loading Up the Simulator Package

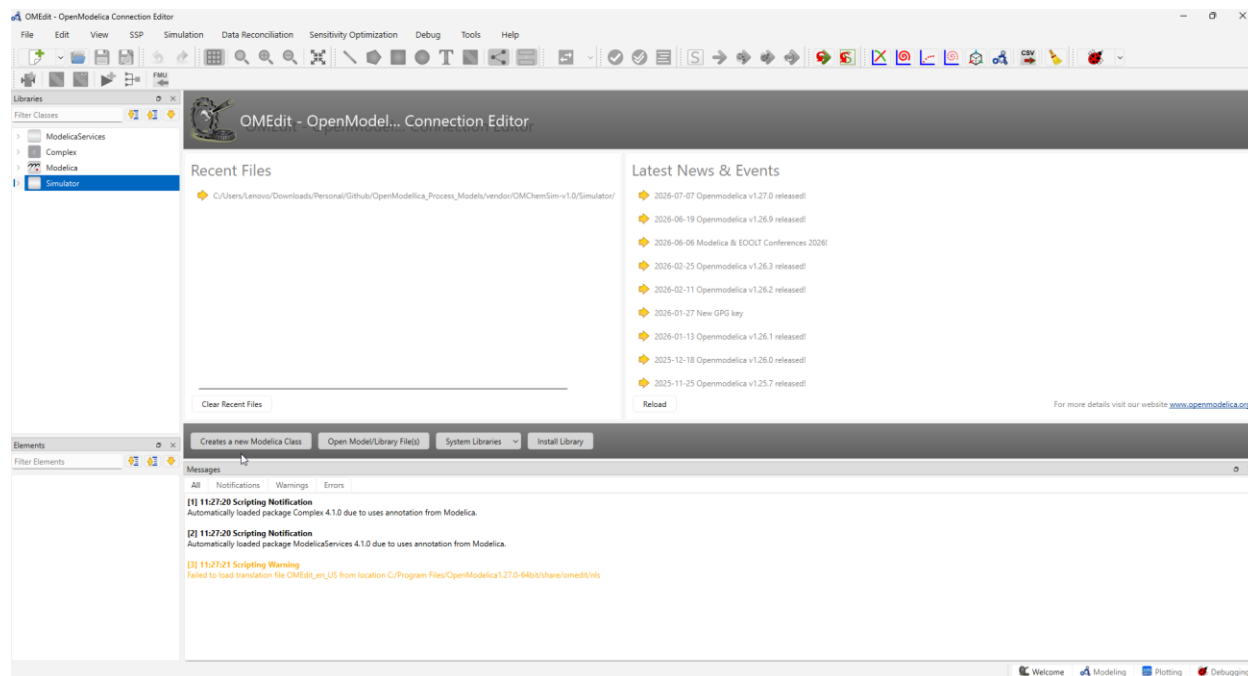


Figure 2 OMEdit After Loading Up the Simulator Package

This single action loads the entire library into your OMEdit session not just the one file, but every sub-package beneath it: streams, compound properties, thermodynamic packages, unit

operations, and the internal guess-value system. You'll see Simulator appear as a new top-level entry in the libraries panel on the left, the same panel you used in the first guide to browse the Modelica Standard Library.

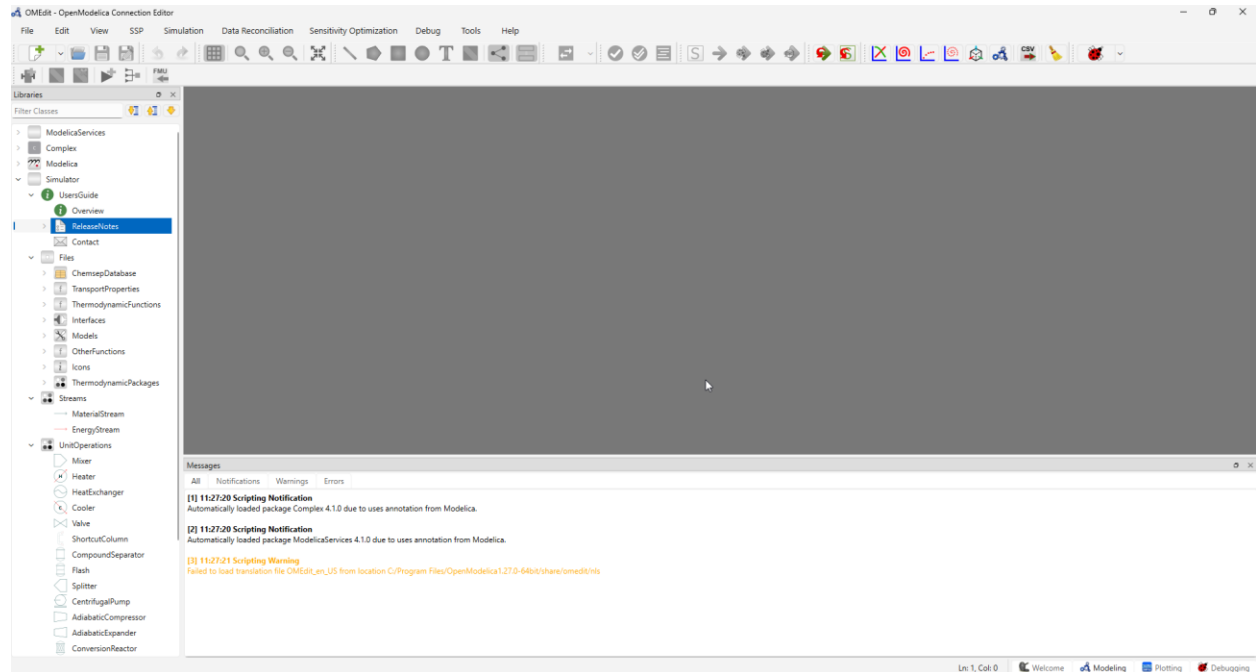


Figure 3 OMEdit With Simulator Tree View

2.4 SETTING UP YOUR OWN WORKSPACE MODEL

Before writing anything, resist the temptation to open files inside **Simulator** and start editing. Treat the library the same way you'd treat a vendor-supplied unit operation in DWSim or a licensed property package in Aspen: something you build on top of, not something you modify in place. Any changes you make directly inside the library are easy to lose, and they blur the line between "code I wrote" and "code the library shipped with," which makes troubleshooting far harder later.

Instead, create a separate package of your own, alongside **Simulator** rather than inside it. In OMEdit, right-click in the Libraries panel and choose New Modelica Class, give it a name (this guide uses **MyModels**, matching the convention already used in this repository's own numbered example folders), and set its type to Package. This new package is where every model you build in the rest of this guide will live.

2.5 VERIFYING THE IMPORT

Before moving on, take a moment to confirm the import succeeded. Expand the **Simulator** entry in the libraries tree and check that you can see its major sub-packages: **Streams**, **Files**, **UnitOperations**, and **GuessModels** should all be visible. As a further sanity check, drill into **Simulator.Streams** and confirm **MaterialStream** is listed, or into **Simulator.Files.ChemsepDatabase** and confirm you can see a long alphabetical list of

compounds **Water**, **Ethanol**, and hundreds of others.

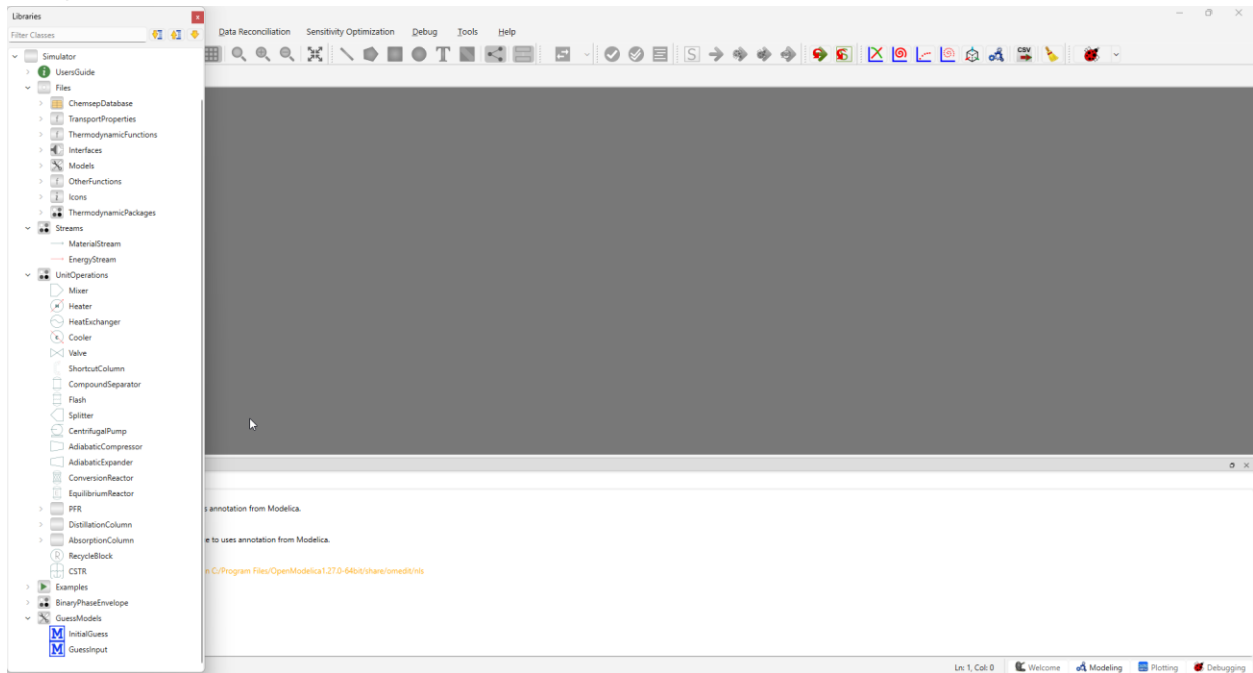


Figure 4 Simulator Package Extended

If all of that is visible, the library is fully loaded and you're ready to start building.

3 UNDERSTANDING THE KEY BUILDING BLOCKS

Before writing your first model, it's worth spending a few minutes understanding what you're assembling. This chapter is a map, not a manual short, conceptual descriptions of the pieces you'll use in Chapter 4, so that when you see **extends Simulator.Streams.MaterialStream(...)** in a code block, you already know roughly what that line is asking for.

3.1 SIMULATOR.STREAMS.MATERIALSTREAM

If you've used DWSim or Aspen, "material stream" probably recalls something fairly passive: a labelled arrow on a flowsheet, carrying a flow rate, composition, temperature, and pressure from one unit operation to the next. **Simulator.Streams.MaterialStream** is not that. It's a genuinely active, flash-capable model. Give it temperature, pressure, a composition, and a flow rate, and it will compute on its own, using whatever thermodynamic package you pair it with its own bubble point, dew point, and vapor/liquid phase split.

That distinction matters how you use it. You won't need a separate "flash drum" unit operation just to find out whether a stream is subcooled liquid, superheated vapor, or sitting inside the two-phase envelope; the stream itself already knows.

3.2 SIMULATOR.FILES.CHEMSEPDATABASE

Every compound a stream can be built from water, ethanol, propane, and 431 others lives in this package, one file per compound, each one a record of that substance's physical properties: critical temperature and pressure, vapor pressure correlation coefficients, heat capacity polynomials, and more, all originally sourced from Chemsep. Rather than typing these numbers in yourself, you reference a compound by name (**data.Water**, **data.Ethanol**) and the correct property set comes with it.

It's worth knowing what this database is not, something we'll return to properly in Chapter 6. These are general-purpose correlations suited to multicomponent mixtures, not the specialized, reference-quality formulations you'd use for a single pure substance under demanding conditions. For an ordinary hydrocarbon or organic compound in a mixture, **ChemsepDatabase** is exactly the right tool.

3.3 SIMULATOR.FILES.THERMODYNAMICPACKAGES

A material stream on its own doesn't know how to relate composition, temperature, and pressure to phase behavior that's the job of a thermodynamic package, sometimes called a property method or property package in DWSim and Aspen. OMChemSim ships six: **Raoult'sLaw**, **NRTL**, **UNIQUAC**, **UNIFAC**, **PengRobinson**, and **GraysonStreed**. Each one embodies a different set of assumptions about how real liquid and vapor phases are allowed to be, and each is suited to different kinds of systems:

Table 1 Thermo-Packages

Package	Best suited for	Key assumption
Raoult's Law	Ideal or near-ideal mixtures; a good starting point for learning	Liquid and vapor are both treated as ideal (activity and fugacity coefficients = 1)
NRTL	Non-ideal liquids, including systems with azeotropes	Local-composition activity coefficient model
UNIQUAC	Non-ideal liquids, especially mixtures of molecules with different sizes and shapes	Combinatorial (size/shape) plus residual (energetic) activity terms
UNIFAC	Non-ideal liquids when you don't have measured binary interaction data	Estimates UNIQUAC-like behavior from molecular functional groups
Peng-Robinson	Hydrocarbons and light gases, including at elevated pressure	Cubic equation of state applied consistently to both phases
Grayson-Streed	Hydrocarbon systems containing hydrogen, common in refinery applications	Empirical correlation tuned for hydrogen/hydrocarbon K-values

You'll use Raoult's Law for the first model in this guide, precisely because its assumptions are the simplest to reason about, and Peng-Robinson later in Chapter 6, where a genuinely non-ideal hydrocarbon system calls for it.

One structural detail worth knowing before you reach Chapter 4: every one of these six packages, along with **MaterialStream** itself, is built on a shared foundation called **PartialThermolInterface**. In plain terms, a material stream and its thermodynamic package need to agree on a common set of variables equilibrium K-values, residual enthalpy, and so on with the stream consuming them and the thermodynamic package producing them. **PartialThermolInterface** is where those shared variables are declared exactly once, so that both sides of that relationship are always talking about the same thing. You won't need to open or edit this file directly, but you will see it mentioned again in Chapter 5, since a mismatch of exactly this kind was one of the more instructive bugs we found while building this guide.

3.4 SIMULATOR.FILES.INTERFACES.MATCONN

matConn is the connector every material stream uses for its **In** and **Out** ports the equivalent of a stream connection arrow in DWSim's flowsheet view. It carries pressure, temperature, total flow, enthalpy, entropy, phase-wise composition, and vapour fraction between whatever it's attached to. For the single standalone stream you'll build in Chapter 4, you won't need to connect anything to these ports at all that becomes relevant once you start linking streams into unit operations, which is beyond the scope of this guide.

3.5 SIMULATOR.GUESSMODELS

Every flash calculation a material stream performs is solved iteratively, and every iterative solver needs a starting point. **GuessModels** is the part of the library responsible for generating those starting points automatically reasonable first estimates for things like bubble and dew temperature, vapor fraction, and phase composition, computed before the real solve begins.

For the models in this guide, you won't need to touch anything inside **GuessModels** directly; it works quietly in the background every time you extend **MaterialStream**. It's mentioned here mainly so that if you ever see it referenced in an error message, you'll recognize it as part of the initial-guess machinery rather than something foreign. We'll come back to exactly how that machinery can go wrong and how to fix it when it does in Chapter 5.

3.6 SIMULATOR.UNITOPERATIONS

Beyond streams, OMChemSim also ships a substantial set of unit operations: heaters, coolers, mixers, splitters, valves, pumps, compressors and expanders, flash drums, CSTR and PFR reactors, and distillation and absorption columns. Every one of them is built the same way a material stream is by extending the same shared guess-value machinery described above, and expecting to be connected to streams through **matConn** ports.

This guide doesn't cover unit operations hands-on; our scope here is deliberately limited to building and understanding streams on their own. But everything in this chapter and the next transfers directly once you're ready to move on to them the same compound database, the same thermodynamic packages, the same connector, and, as it happens, several of the same categories of issue you'll learn to recognize in Chapter 5.

4 BUILDING A PROCESS MODEL FROM SCRATCH USING THE SIMULATOR PACKAGE

This is the chapter where everything from Chapters 1 through 3 comes together into a working model. We're going to build one thing a single material stream, standing on its own, with no unit operations attached and understand every line of it before moving on. Keeping the scope this narrow is deliberate: a standalone stream is the smallest possible unit of "real" chemistry you can build in OMChemSim, and it exercises the entire chain (compound database → thermodynamic package → material stream) without anything else to distract from it.

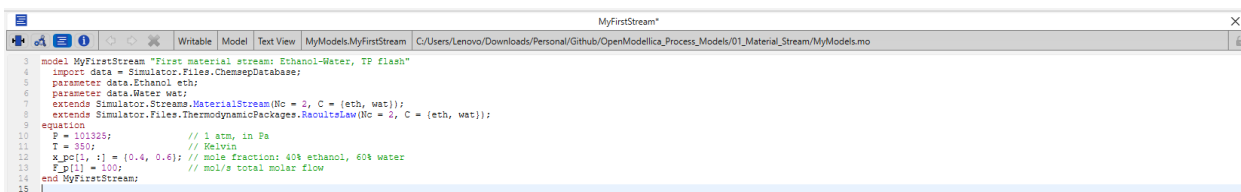
4.1 CHOOSING A SYSTEM

We'll model a binary mixture of **ethanol and water**. This isn't an arbitrary choice. It's a real, well-studied VLE pair one you've almost certainly worked with before if you've done any distillation-related work in DWSim or Aspen and it's genuinely non-ideal, which matters for a reason we'll return to shortly. At the same time, it's simple enough (two components, no reactions, no recycle) that if something doesn't behave the way you expect, you'll be able to reason about why without a large system getting in the way.

One honest caveat, worth stating up front rather than discovering later: this guide pairs the stream with Raoult's Law (Section 3.3), which assumes ideal liquid and vapor behavior. Ethanol and water are known to deviate from that assumption meaningfully they form a well-known azeotrope so treat the results in this chapter as a reasonable first approximation, not a precise prediction. If you wanted an accurate azeotrope prediction for this specific pair, NRTL or UNIQUAC would be the correct choice instead. We're using Raoult's Law here purely because its simplicity makes it the right tool for learning the workflow, which is this chapter's actual goal.

4.2 WRITING MYFIRSTSTREAM.MO

Inside the **MyModels** package you created in Chapter 2, create a new model called **MyFirstStream**, switch to Text View, and enter the following:



```
1 model MyFirstStream "First material stream: Ethanol-Water, TP flash"
2   import data = Simulator.Files.ChemsepDatabase;
3   parameter data.Ethanol eth;
4   parameter data.Water wat;
5   extends Simulator.Streams.MaterialStream(Nc = 2, C = {eth, wat});
6   extends Simulator.Files.ThermodynamicPackages.RaoultLaw(Nc = 2, C = {eth, wat});
7
8   equation
9     P = 101325; // 1 atm, in Pa
10    T = 350; // Kelvin
11    x_pc[1, :] = {0.4, 0.6}; // mole fraction: 40% ethanol, 60% water
12    F_p[1] = 100; // mol/s total molar flow
13  end MyFirstStream;
```

Figure 5 Code for Material Stream

That's a complete, working material stream in eleven lines. Let's take it apart.

4.3 LINE-BY-LINE EXPLANATION

If you have DWSim or Aspen experience, the table below maps each line onto something you've almost certainly done before in those tools the actions are the same, only the interface has changed.

Table 2 Code Line Explanation

Line	Role	DWSim/Aspen Equivalent
<code>import data = Simulator.Files.ChemsepDatabase;</code>	Creates a short alias, data, so the two lines below don't need to spell out the full package path	-- (a Modelica convenience, no direct equivalent)
<code>parameter data.Ethanol eth; / parameter data.Water wat;</code>	Pulls in the property record for each compound from the compound database	Selecting compounds in the component list
<code>extends Simulator.Streams.MaterialStream(Nc = 2, C = {eth, wat})</code>	Instantiates the full stream model with exactly these two compounds	Adding a Material Stream object to the flowsheet
<code>extends Simulator.Files.ThermodynamicPackages.RaoultzLaw(Nc = 2, C = {eth, wat})</code>	Selects the VLE method the stream will use to flash	Selecting a property/fluid package
<code>P = 101325; / T = 350;</code>	Your two flash specifications -- pressure and temperature	Entering T and P on the stream's property tab
<code>x_pc[1, :] = {0.4, 0.6};</code>	Feed composition -- index 1 refers to the overall (mixed) phase	The composition tab
<code>F_p[1] = 100;</code>	Total molar flow rate	The flow rate field

One detail worth flagging now, even though the full explanation belongs in the next chapter: notice that **(Nc = 2, C = {eth, wat})** appears on both **extends** lines, not just the first. This isn't redundant copy-paste -- it's required, and leaving it off the second line produces a real error. Chapter 5

explains exactly why, once you've seen the underlying architecture that makes it necessary.

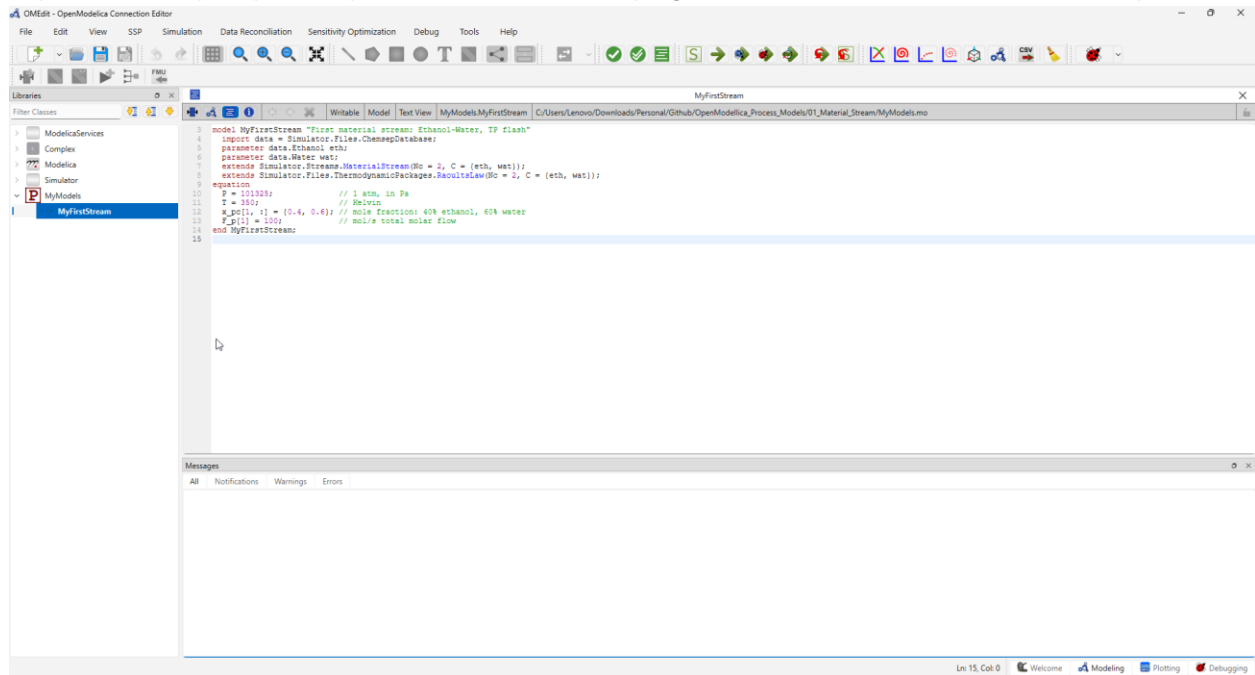


Figure 6 MyFirstStream Model Code

4.4 CHECK MODEL

With **MyFirstStream** written and saved, run **Check Model** the same way you did with the simple ramp model in the first guide. This step doesn't run a simulation, it verifies that the model is

internally consistent: that every unknown has exactly one equation to solve it, with nothing missing and nothing left over.

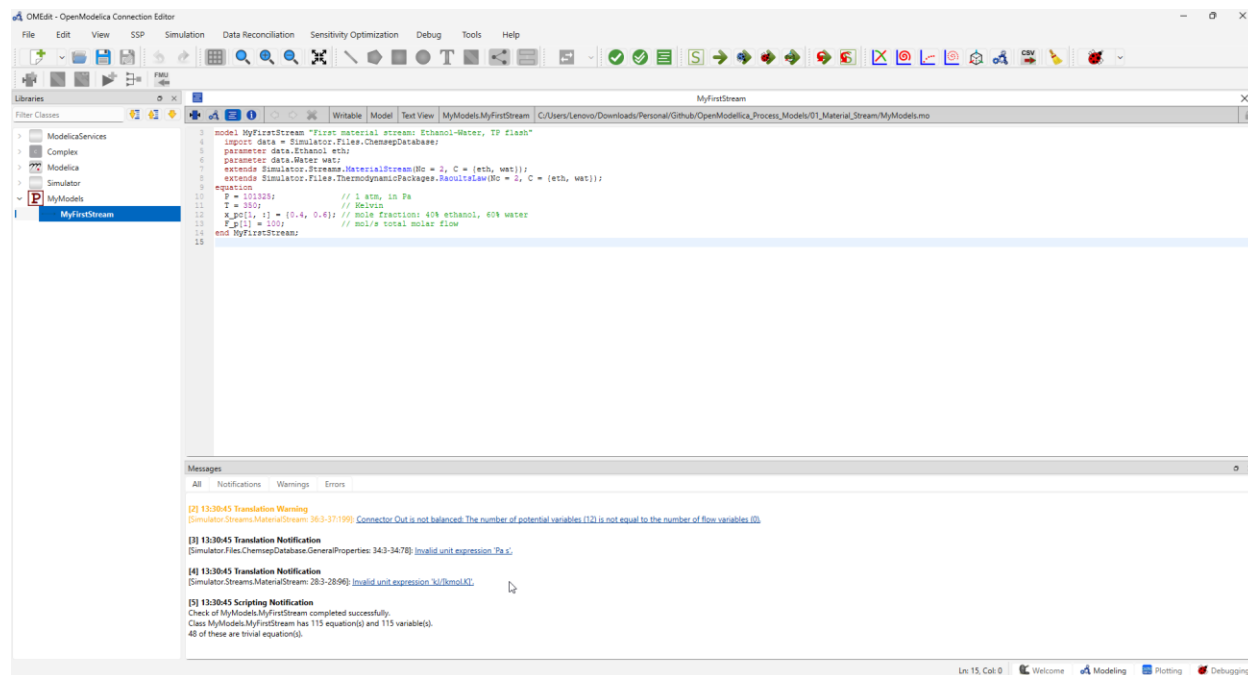


Figure 7 Model Check

For a correctly built stream like this one, you should see a message reporting a specific count of equations and variables and, critically, that the two numbers match. A material stream is a substantially larger system than the ramp function from Doc 1 (this one resolve to well over a hundred equations once the compound properties and thermodynamic package are fully accounted for), but the principle is identical: equations must balance variables, or the model cannot be solved.

You'll also likely see the check report that some number of those equations are "trivial" direct assignments the solver can resolve immediately, without iteration. That's a good sign, not a concern; it simply means part of the system is straightforward bookkeeping rather than genuine simultaneous equations.

If your check comes back clean, you're ready to move on to Chapter 5 which, fittingly, is about what to do when it doesn't.

5 TROUBLESHOOTING: READING OPENMODELICA ERRORS

If Chapter 4 went smoothly for you, this chapter might feel unnecessary at first glance. It isn't. Building the models in this guide surfaced six distinct, real bugs in OMChemSim, not contrived examples, but the actual errors encountered while writing this material and every one of them is worth understanding, because the patterns behind them show up constantly in Modelica work generally, well beyond this one library. This chapter teaches you to recognize those patterns. The

full technical fix for each one lives in **vendor/OMChemSim-v1.0/ATTRIBUTION.md**; what belongs here is how to *think* about an error when you see one.

5.1 WHY ERRORS ARE NORMAL HERE

It's worth saying plainly: if you follow this guide using an unpatched copy of OMChemSim, you will very likely hit errors, and that is not a sign you've done something wrong. As discussed in Chapter 1, this library was almost certainly built and tested against an older, more permissive version of the OpenModelica compiler. Today's compiler enforces parts of the Modelica language specification more strictly than it once did, and several structural issues in the library's core files, the ones every single model depends on only surface under that stricter enforcement.

This matters for how you should react when something breaks. An error here is far more often a genuine library issue than a mistake in a few lines of your own code, which is the opposite of what most of your prior simulator experience has probably trained you to assume. Treat each error as a diagnostic puzzle to work through methodically, not as evidence you've misunderstood something fundamental.

5.2 HOW TO READ A TRANSLATION ERROR

Every OpenModelica error follows the same basic shape:

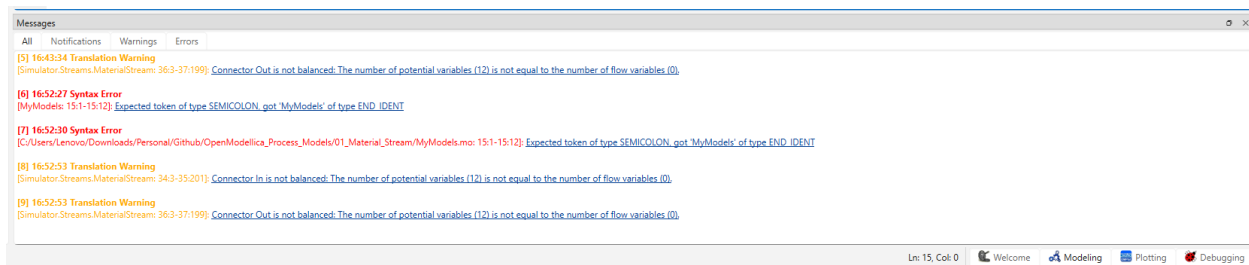


Figure 8 Errors in Openmodelica

Three pieces of information matter here, in order of importance:

1. **The file name in square brackets** tells you *where the problem physically lives* not necessarily the model you're building, but wherever the offending code is actually written. If you're checking **MyFirstStream** and the error names **Simulator.GuessModels.InitialGuess**, the problem lives inside that library file, even though you never opened it.
2. **The line and column numbers** pinpoint the exact location within that file open it and go straight there rather than reading the whole thing top to bottom.
3. **The description** tells you what rule was violated. This is often written in fairly literal, low-level compiler language rather than plain English, which is exactly what Section 5.3 exists to translate.

As a concrete example: an error reading **[Simulator.GuessModels.InitialGuess: 45:6-47:13]: Variable Nc not found in scope InitialGuess**. is telling you, precisely, that somewhere in

InitialGuess.mo, between line 45 and line 47, the compiler encountered a reference to **Nc** and could not find any declaration of it reachable from that file's own class hierarchy. Once you can read that sentence confidently, you're most of the way to diagnosing the six categories below.

5.3 COMMON ERRORS YOU'LL HIT

5.3.1 Broken imports.

Look for an error like **Import Simulator.Files.Thermodynamic_Functions not found in scope <top>**. This means an **import** statement references a package name that doesn't actually exist often due to a small typo (an extra underscore, wrong capitalization) that's easy to miss on a read-through. The fix is either correcting the name or, if nothing in the file actually relies on the unqualified import, simply removing the broken line.

5.3.2 **model** vs. **record** specialization mismatch.

Look for **Component 'C' may not have a binding equation due to class specialization 'model'**. Modelica has a dedicated class type, **record**, for pure data holders things with only **parameter** declarations and no equations. When a class like this is declared with the **model** keyword instead, you can't bind its array elements the way **extends SomeStream(C = {eth, wat})** requires. If you see this error on a class that clearly holds nothing but property data, check whether it's declared **model** where it should say **record**.

5.3.3 Missing **partial** on template/mixin classes.

Look for **Variable X not found in scope SomeClass**, where **X** is used throughout the file but never declared there. Some classes are deliberately incomplete written to have certain variables supplied later, by whatever class extends them. Modelica requires such a class to be marked **partial** to say so explicitly; without it, the compiler correctly refuses to accept a class that references something it doesn't itself define.

5.3.4 Cross-class variable scoping.

This is the deepest and most important pattern in this list, and it's worth understanding properly rather than just recognizing the error text. Modelica name resolution only looks upward, through a class's own ancestors never downward into a class that later extends it, and never sideways into an unrelated sibling class combined with it only by a third, further-derived class. This explains two things you may have already noticed in Chapter 4: why **Nc/C** sometimes need to be declared in a shared common ancestor rather than duplicated locally, and why **(Nc = 2, C = {eth, wat})** had to appear on both **extends** lines in **MyFirstStream.mo** when the same ancestor is reached through two separate paths, Modelica requires every path to apply an identical modification, or you'll see **Duplicate elements (due to inherited elements) not identical**. If an error names a variable that's declared elsewhere in the library but not in the reporting class's own ancestry, this is almost always the cause.

5.3.5 Missing **each** on array-valued unit attributes.

Look for **Non-array modification "... " for array component 'unit', possibly due to missing 'each'**. When an attribute like **unit=** is meant to apply to every element of an array parameter, Modelica requires the **each** keyword to say so explicitly (**each unit = "Pa"**), not (**unit = "Pa"**).

Without it, the compiler tries to bind a single scalar attribute to an entire array at once, which isn't valid.

5.3.6 Nonlinear initialization failures.

These look different from the other five not a translation error at all, but a simulation-time failure such as **division by zero at time 0** or repeated **Homotopy solver total pivot: Matrix (nearly) singular at initialization** warnings. This category is fundamentally different in kind: the model is correct, but the iterative solver's starting guess is too far from the true answer to converge. It's the same category of problem as a divergent initial estimate in Aspen or DWSim, and the fix is the same in spirit supply better starting values with one important nuance specific to this library: OMChemSim's shipped default guesses are often tuned to whatever compounds its original authors happened to test with, and can be badly wrong for a different compound pair. Recognizing this category matters because the fix isn't a code correction at all, it's a numerics one.

5.4 WHEN TO SUSPECT THE LIBRARY VS. YOUR OWN CODE

A simple rule of thumb: if an error names a file inside **vendor/OMChemSim-v1.0/Simulator/** that you haven't personally written or edited, treat the library as the first suspect, not your own model. This repository's copy of OMChemSim has already been patched for every issue described above in the files this guide actually uses so if you're following along exactly as written and still see one of these patterns, check **ATTRIBUTION.md** first; it may already be documented, or it may be a new instance of the same category of issue in a file we haven't touched yet (unit operations and the less-common thermodynamic packages are the most likely candidates, simply because this guide hasn't exercised them as thoroughly).

If, on the other hand, an error names your own file **MyModels.mo** or whatever you've called it the more likely culprit is an ordinary syntax slip: a missing semicolon, a mismatched bracket, a modifier applied inconsistently across two **extends** lines as described above. Start there before assuming something deeper is wrong.

Either way, the same general approach works read the file and line the error names, work out which of the six patterns above it resembles, and fix the narrowest thing that addresses it rather than guessing broadly or changing things that aren't implicated.

6 DECLARING OTHER STREAMS IN THE SAME MODEL

Everything so far has used one stream, one thermodynamic package, one set of compounds. This chapter goes further, using a companion model file (**03_Multi_Stream/MultiStream.mo**) that puts two genuinely different kinds of stream side by side in a single model: a multicomponent hydrocarbon mixture flashed with Peng-Robinson, and a pure water stream evaluated on the Modelica Standard Library's own steam tables. This is a richer example than a second simple stream, and it surfaces several things worth understanding properly before you build anything like it yourself.

6.1 OVERVIEW: TWO STREAMS, TWO PROPERTY METHODS, ONE MODEL

Everything a stream needs comes from one of two places, and this model draws on both at once. OMChemSim, as covered in Chapters 1 and 3, supplies **MaterialStream** and its thermodynamic packages for genuine multicomponent mixtures, composition is the entire point of it, along with the VLE, K-values, and bubble/dew pressures that come with it. The Modelica Standard Library, separately, supplies **Modelica.Media**, a family of medium models covering water, air, and various refrigerants, of which **Water.StandardWater** (the IAPWS-IF97 steam tables) is one. A medium model like this has no composition at all, because it doesn't need one; what it offers instead is reference-quality property data for a single, well-defined substance across the full range a real steam system might use.

Neither of these subsumes the other. Peng-Robinson gives poor water properties; the steam tables cannot flash a hydrocarbon mixture. A real flowsheet has both kinds of stream in it side by side, and this model is built to demonstrate exactly that.

One practical note before you load this model yourself: combining OMChemSim with the Modelica Standard Library in the same OEdit session means load order matters. The specifics are documented in **03_Multi_Stream/README.md** follow that before opening **MultiStream.mo** for the first time.

6.2 COMPOSITES: JOINING A STREAM TO A PROPERTY PACKAGE

A **MaterialStream** on its own is incomplete as you saw in Chapters 3 and 4, it consumes K-values and residual properties that a thermodynamic package has to supply. The **Composites** package captures that pairing once, cleanly, so it doesn't need to be repeated inline every time:

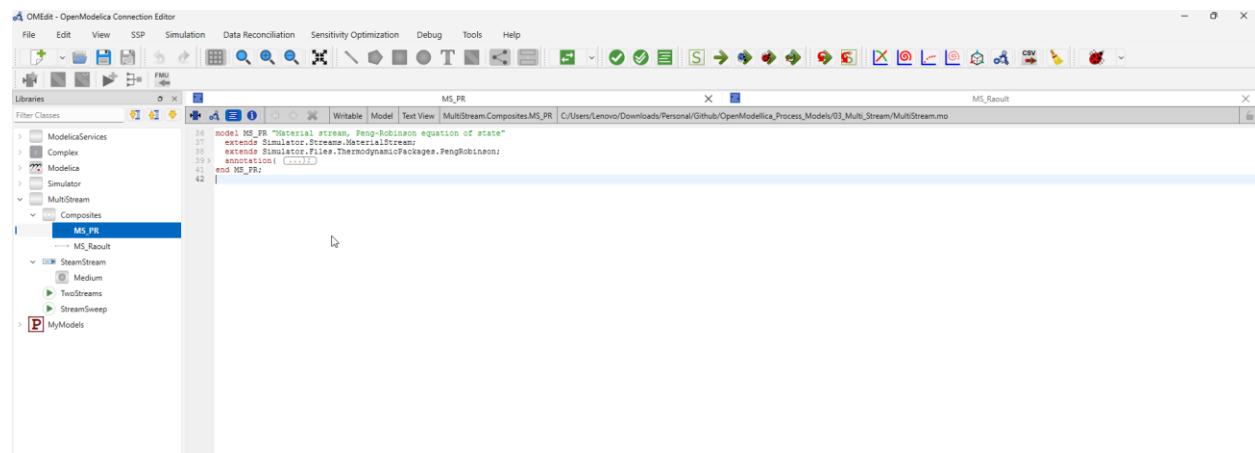


Figure 9 Peng Robinson Material Stream

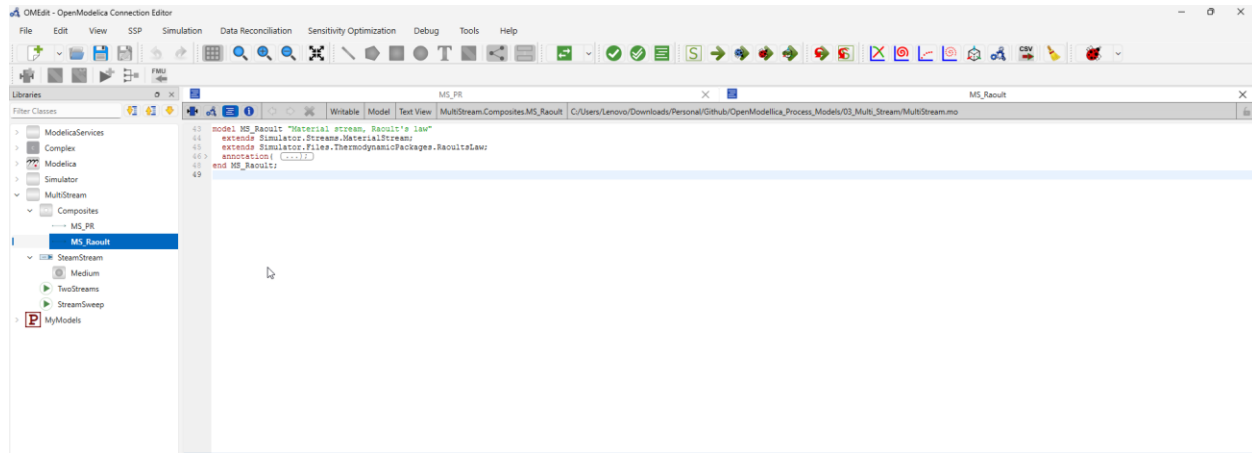


Figure 10 Raoult's Law Material Stream

Notice what's *missing* here compared with **MyFirstStream.mo** in Chapter 4: neither **Nc** nor **C** is set on either **extends** line. That's intentional, and it follows directly from the scoping rule covered in Chapter 5 modifying **Nc/C** on only one of two **extends** clauses that both reach the same shared ancestor produces an error, since every path to that ancestor must apply an identical modification. The way around that here is to leave both unmodified in the composite itself and specify the actual components only at the point of use, as you'll see in Section 6.3 (**HC(Nc = Nc, C = C, ...)**). This is a cleaner pattern than what we did in Chapter 4, and it's worth adopting for your own future models: build the "stream + property package" pairing once, generically, and supply the specific compounds only where the composite is actually instantiated.

6.3 A HYDROCARBON STREAM VIA PENG-ROBINSON (**HC**)

The hydrocarbon stream models a light NGL (natural gas liquids) cut propane, n-butane, and n-pentane, in roughly a 50/30/20 mole-percent split. This is exactly the kind of system Section 3.3's guidance table pointed toward Peng-Robinson for non-polar hydrocarbons where a cubic equation of state, applied consistently to both phases, captures the real-gas behavior that Raoult's Law simply assumes away.

This stream also demonstrates something worth knowing before you build your own multicomponent flash: Peng-Robinson is considerably less forgiving of a poor initial guess than Raoult's Law. Left to the generic, equimolar-feed assumption that **InitialGuess** derives automatically (Section 3.5), a Peng-Robinson flash on a genuinely skewed feed composition can either fail to initialize at all, or more subtly troubling converge on a mathematically valid root of the underlying vapor-liquid split equation that lies outside the physically meaningful range of zero to one, and so isn't a real vapor fraction at all. Both failure modes are start-value problems, in the same sense discussed in Chapter 5: the equations are correct, but the solver's starting point matters more here than it did for the simpler system in Chapter 4.

The fix demonstrated in this model is to seed the guess system directly, using three parameters **Pg**, **Tg_user**, and **xg_user** set to the same values as the stream's actual specification, plus an explicit **start** value on **xvap** itself:

```

167  --- The two streams -----
168  /* Pg, Tg_user and xg_user seed the start values from the same parameters
169  that carry the specification, so an overridden operating point gets an
170  initial guess that follows it instead of one frozen at the defaults. */
171  Composites.MS_PR HC (Nc = Nc, C = C, Pg = P_hc, Tg_user = T_hc,
172  xg_user = {z_c3, z_nc4, 1 - z_c3 - z_nc4},
173  xvap(start = xvap_guess))
174  "Hydrocarbon stream, Peng-Robinson"
175  annotation(Placement(visible = true, transformation(origin = {0, 40}, extent = {{-20, -20}, {20, 20}}, rotation = 0)));
176
177  SteamStream W "Water stream, IF97 steam tables"
178  annotation(Placement(visible = true, transformation(origin = {0, -40}, extent = {{-20, -20}, {20, 20}}, rotation = 0)));
179

```

Figure 11 Fix of Tg, Pg, and Xg

The effect is that if you change the stream's operating point, its initial guess follows that change automatically, rather than staying frozen at whatever generic default the library ships with precisely the same category of fix as the compound-specific **start** values from Chapter 5, applied here at the point of use instead of inside the library itself.

6.4 A WATER STREAM VIA THE IAPWS-IF97 STEAM TABLES (W)

SteamStream is deliberately shaped to match the pattern you already know from an OMChemSim stream: it declares **P**, **T**, and **F**, leaves them unsolved, and expects the enclosing model to specify them exactly the same relationship as **HC.P**, **HC.T**, and **HC.F_p[1]** in Section 6.3. Everything else density, enthalpy, entropy, specific heat, and where the stream sits relative to saturation is computed from those three inputs via **Modelica.Media.Water.StandardWater**.

The choice to use the Standard Library's steam tables here, rather than water's entry in **ChemsepDatabase**, is deliberate and worth restating from Section 3.2: for a single pure substance under demanding conditions, reference-quality property data matters more than composition flexibility, and that's exactly what IAPWS-IF97 provides.

One physical caveat is worth understanding clearly before you experiment with this model yourself: a pressure-temperature pair cannot specify a genuinely two-phase state. Exactly on the saturation line, pressure and temperature stop being independent of each other water boiling at a given pressure has exactly one saturation temperature, so specifying both doesn't add information, it creates an inconsistency. A stream that needs to represent a true two-phase condition has to be specified differently, using pressure paired with enthalpy or vapor fraction instead. For the purposes of this guide, the practical takeaway is simpler: keep (**P**, **T**) specifications away from the saturation line and expect properties to change abruptly if a swept point crosses it -- which becomes directly relevant again in Section 6.8.

6.5 WATCH THIS: UNIT MISMATCHES BETWEEN LIBRARIES

Here is a genuinely subtle trap, and one that's worth internalizing as a general lesson rather than a one-off fix. OMChemSim carries molar flow in mol/s and molecular weight in kg/kmol numerically identical to g/mol. Multiply the two together, as **MaterialStream** does internally to compute **Fm_p**, and the result is numerically grams per second, even though the variable's declared unit string says **kg/s**. **SteamStream.Fm**, by contrast, is genuine kg/s, because its own calculation explicitly divides by 1000 to convert.

The consequence: adding **HC.Fm_p[1]** directly to **W.Fm** would silently combine a number in grams/s with a number in kilograms/s, producing a total that's wrong by a factor of a thousand, with

no error or warning from OpenModelica at all unit declarations are documentation for the reader, not something the compiler verifies against the actual arithmetic. **TwoStreams** handles this correctly by converting explicitly before combining:

```

190   HC.P = P_hc;
191   HC.T = T_hc;
192   HC.F_p[1] = F_hc;
193   HC.x_pc[1, :] = {z_c3, z_nc4, 1 - z_c3 - z_nc4};
194
195   W.P = P_w;
196   W.T = T_w;
197   W.F = F_w;
198
199   Fm_hc = HC.Fm_p[1] / 1000;
200   Fm_total = Fm_hc + W.Fm;
201
202 >   annotation( ... );
203 end TwoStreams;
204
205
206

```

Figure 12 handling of 2 streams

The general lesson matters well beyond this one case: whenever two different libraries meet inside one model, treat every declared unit as a claim to verify, not a fact to trust. Trace back to where a quantity is actually computed and confirm the numbers, especially the moment before you combine values that originated from two different sources.

6.6 REFERENCE-STATE CAUTION

A related but distinct trap concerns enthalpy and entropy specifically. IAPWS-IF97 sets both to zero at the saturated liquid triple point a fixed, universal reference. OMChemSim, by contrast, builds its enthalpy values from standard heats of formation, an entirely different zero point. The consequence is that **SteamStream.H_molar** and an OMChemSim stream's **H_p[1]** are simply not comparable as absolute numbers there is no meaningful physical interpretation of subtracting one from the other directly.

What *is* valid is comparing a difference between two states measured on the *same* reference framework two **SteamStream** instances at different conditions, for instance, to compute a genuine enthalpy change across a heater or cooler. The rule to carry forward: absolute enthalpy and entropy values are only ever meaningful relative to whatever reference state produced them, and mixing reference states between two different property calculations, even inadvertently, produces numbers that look plausible but mean nothing.

6.7 TWOSTREAMS: RUNNING BOTH SIDE BY SIDE

With both streams and their caveats understood, **TwoStreams** simply specifies each one's operating point and lets both solve independently:

```

190   HC.P = P_hc;
191   HC.T = T_hc;
192   HC.F_p[1] = F_hc;
193   HC.x_pc[1, :] = {z_c3, z_nc4, 1 - z_c3 - z_nc4};
194
195   W.P = P_w;
196   W.T = T_w;
197   W.F = F_w;
198

```

Figure 13.2 streams operating point solved independently

Once simulated, the two streams are worth inspecting in parallel rather than in isolation: **HC.xvap**, **HC.K_c[:]**, **HC.Pbubl**, and **HC.Pdew** characterize the hydrocarbon flash, while **W.h**, **W.d**, **W.Tsat**, and **W.dTsup** characterize the water stream's thermodynamic state and how far it sits from saturation.

It's worth being clear about what kind of simulation this actually is. Neither stream contains a time derivative anywhere there's no **der(...)** in either one so despite running through OMEdit's Simulate button, this is not a dynamic simulation at all. It's a single steady-state algebraic solve, and the stop time you choose for it is essentially arbitrary; any value will produce the same answer.

6.8 BONUS: WALKING THE OPERATING POINT OVER SIMULATED TIME (**STREAMSWEEP**)

Everything up to this point could, in principle, be done in a sequential-modular tool too the specific implementation differs, but the concept of a multicomponent flash beside a steam-table stream isn't unique to OpenModelica. This last model is different. It's the closest thing in this guide to a genuine demonstration of what an equation-based, acausal environment offers that a traditional flowsheet simulator structurally cannot.

The idea is simple to state: reuse **time** itself, OpenModelica's built-in simulation clock from Doc 1, as the coordinate you sweep an operating point across.

```

274   tau = (time - t_start) / (t_stop - t_start);
275
276   HC.T = T_hc_start + tau * (T_hc_end - T_hc_start);
277   HC.P = P_hc_start + tau * (P_hc_end - P_hc_start);

```

Figure 14 Setting up time function

It's worth being precise about what this is and isn't. Because neither stream has a time derivative, this is still not dynamic simulation in the usual sense it's a *sequence of independent steady-state solves*, one per output interval, with the specification nudged slightly between each one. Thought of that way, it's a legitimate one-factor-at-a-time experiment: fix a **_start/_end** pair for whichever specification you want to vary, hold everything else constant, and the resulting plot is that factor's response curve. As shipped, the model sweeps hydrocarbon temperature from 305 K to 318 K at a constant 5 bar, producing **HC.xvap** moving from roughly 0.46 to 0.87 a boiling curve, generated as a simulation rather than assembled point by point.

Here is the genuinely distinctive part. Each step in that sequence doesn't start from **InitialGuess**'s generic guess the way a cold solve would it starts from the *previous* step's already-converged answer. That continuation is exactly what allows a Peng-Robinson flash, which Section 6.3 already flagged as sensitive to its starting point, to be carried smoothly across the two-phase region

without landing on a spurious, non-physical root. A sequential-modular tool solving each operating point independently has no equivalent mechanism for this; OpenModelica gets it essentially for free, simply because the model is expressed as one continuous system evaluated over a time-like coordinate rather than as a series of unrelated, one-off calculations.

This technique has real limits, and they're worth stating as clearly as the benefit:

- **It sweeps one path, not a grid.** Vary two specification pairs at once and the result is a single diagonal line through the operating space, not an exploration of it. A genuine multi-factor design of experiments needs `tools/run_doe.py` instead of scope for this guide, but worth knowing it exists for when you're ready.
- **Stay inside one phase region.** Driving both temperature and pressure from time keeps all three flash-region branches (subcooled, two-phase, superheated) live simultaneously, and the vendored library does not reliably track a region change partway through a sweep `xvap` can run past its physical bound of 1 and the simulation will stop early. The shipped defaults are chosen to stay inside the two-phase envelope for exactly this reason; widen them past a phase boundary at your own risk.
- **`t_start` and `t_stop` must match the experiment's actual `StartTime/StopTime`.** That pairing is what defines `tau`; setting them inconsistently breaks the mapping between simulated time and the sweep coordinate.

Used within those limits, this is a genuinely useful technique, and one worth remembering as a template for your own future models any specification you'd otherwise sweep manually, point by point, in an external script can instead be expressed this way, directly inside the model itself.

7 RESULTS GENERATION AND EXPORTING PLOTS

Every model built so far has ended the same way: check, simulate, and stop. This chapter finishes the workflow properly, using the same export mechanics you already learned in Doc 1's final chapter, applied to something with actual physical content behind it rather than a generic ramp function.

7.1 SETTING UP THE TEST CASE

For this chapter, use `MultiStream.StreamSweep` from Chapter 6 as the running example it produces a genuinely interesting result (a boiling curve, not a single number), which makes it a far better demonstration of plotting and exporting than a static single-point stream would be. Everything described below works identically for `MyFirstStream` or `TwoStreams` too, if you'd rather export a single operating point instead of a swept curve; the mechanics don't change, only how many rows of data you end up with.

Open `StreamSweep`, confirm the code is as you left it in Section 6.8, and run Simulate with the defaults already set in its `experiment` annotation (`StartTime = 0`, `StopTime = 100`, `Interval = 0.5`). One thing worth keeping in mind here doesn't apply to Doc 1's `SimpleRamp`: in this model, simulated time isn't a physical quantity, it's been deliberately repurposed as the sweep coordinate

tau, exactly as explained in Section 6.8. When you export "against time" from this model, you're really exporting against the swept operating point.

7.2 UNDERSTANDING SIMULATION OUTPUT FILES

As in Doc 1, once the simulation finishes, the raw results exist as a file in your working directory visible in OMEdit's status bar, or reachable directly via the **Open Output File** button in the Messages panel. OpenModelica writes results in **.mat** format by default, the same fast, universally readable format used throughout Doc 1; for **StreamSweep**, expect a file named **StreamSweep_res.mat**.

7.3 EXPORT METHOD 1: THE GRAPHICAL EXPORT (PLOTS)

When the simulation finishes, the Plotting Perspective opens automatically, the same as in Doc 1. The one difference here is what you'll want to plot: rather than a variable against time directly, the physically meaningful view is one variable against another **HC.xvap** against **HC.T**, for instance, to see the boiling curve itself rather than two separate curves against time. OMEdit supports this through its parametric/X-Y plotting option (the exact menu wording varies slightly between versions look for an option to set a custom X-axis variable rather than using time by default). Select **HC.T** as the X variable and **HC.xvap** as the Y variable, and you should see a smooth curve rather than the straight diagonal line from Doc 1's **SimpleRamp** example.

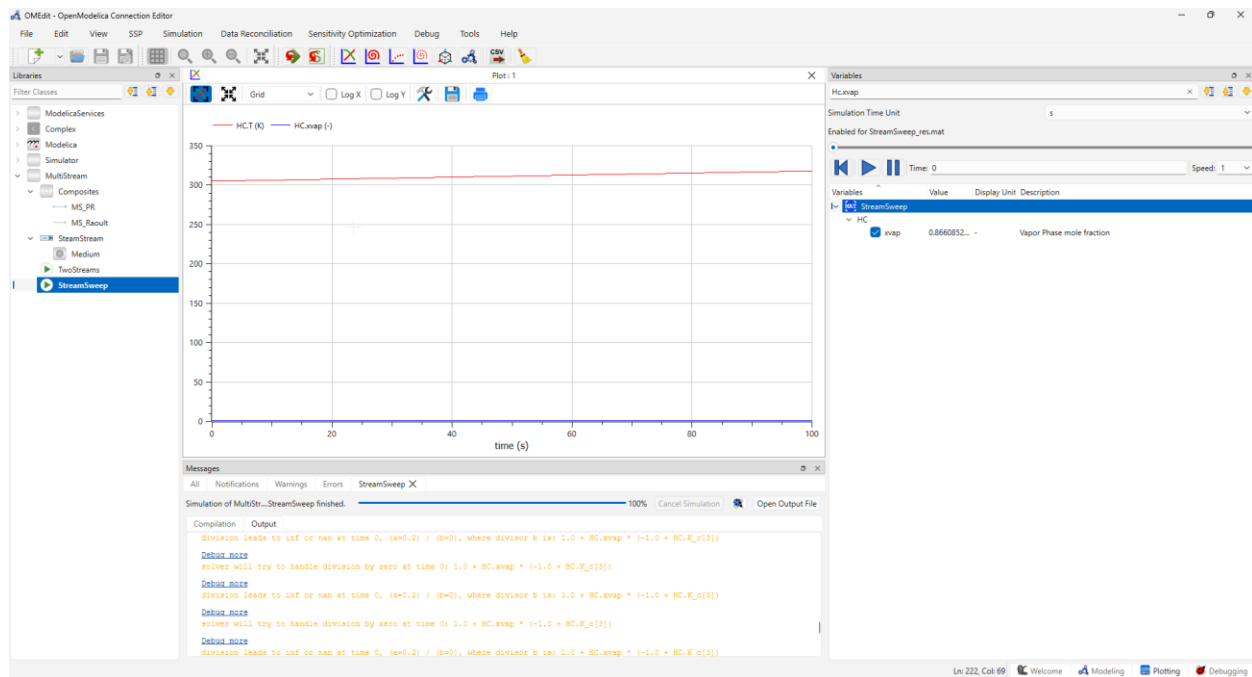


Figure 15 HC.xvap vs. HC.T parametric plot

From there, export exactly as in Doc 1: the export icon in the plot window's toolbar, choose a file name and format (**.png** for quick use, **.svg** if you want a scalable version for a presentation), and save.

7.4 EXPORT METHOD 2: THE CSV (COMMA SEPARATED VALUES) EXPORT

Also unchanged from Doc 1: Simulation → Export → CSV, accept the suggested file name (**StreamSweep_res.csv**), and save. Open the result and you'll find a considerably wider file than **SimpleRamp** produced every variable in the model gets its own column, including **tau**, **HC.T**, **HC.xvap**, **HC.K_c[1:3]**, **HC.Pbubl**, **HC.Pdew**, and the full set of water-stream properties (**W.h**, **W.d**, **W.Tsat**, **W.dTsup**, and more), one row per output interval.

If you set up **OMPpython** following this repository's earlier Python-integration work, this is a good moment to put it to use: loading this CSV into **pandas** and selecting just the columns you actually need (**df[['tau', 'HC.T', 'HC.xvap']]**, for example) is considerably faster than scrolling through dozens of columns in Excel.

7.5 WHAT TO ACTUALLY LOOK AT FOR A MATERIAL STREAM

Doc 1's **SimpleRamp** had exactly one output worth inspecting, **output_value**, and its meaning was whatever you decided it meant. A material stream is different several of its variables carry specific, well-defined physical meaning, and knowing which ones to check tells you something real about the state of the stream:

- **xvap**: the vapor fraction. **0** means subcooled liquid, **1** means superheated vapor, and anything in between means the stream genuinely sits inside the two-phase envelope.
- **Pbubl** and **Pdew**: the bubble and dew point pressures at the stream's current temperature and composition. Compare these against the stream's actual P: if **P** is at or above **Pbubl**, the stream is liquid; if **P** is at or below **Pdew**, it's vapor; anywhere between the two, it's flashing into both phases at once.
- **K_c[:]**: the equilibrium K-value for each component. Values well above 1 indicate a component strongly favoring the vapor phase relative to the mixture as a whole; values well below 1 indicate the opposite. This is your quickest read on relative volatility.
- **x_pc[2,:]** and **x_pc[3,:]**: liquid- and vapor-phase composition respectively, meaningful specifically when the stream is in the two-phase region.

For **StreamSweep** specifically, watching **xvap** climb steadily from roughly 0.46 to 0.87 as **HC.T** runs from 305 K to 318 K is a direct, visual confirmation that the swept operating point is moving through the hydrocarbon feed's two-phase envelope exactly as intended -- the same boiling-curve behavior you'd generate point-by-point in a sequential-modular tool, produced here as a single simulation.

8 REFERENCES AND SOURCE CODES

- [OpenModelica User Guide](#)
- [FOSSEE/OMChemSim \(upstream repo\)](#)
- [This repository \(patched library + CONTRIBUTION.md + example models\)](#)
- [Material Streams Models](#)